

HBF in AI Compute

A System Architect's View

Anurag Agrawal — System Architecture, OXMIQ Labs

Radhakrishna Giduthuri — Software Architecture, PRAXMATI

Hot Chips 2026 · Tutorial

HBF = High-Bandwidth Flash

HBF spec — the hardware metrics

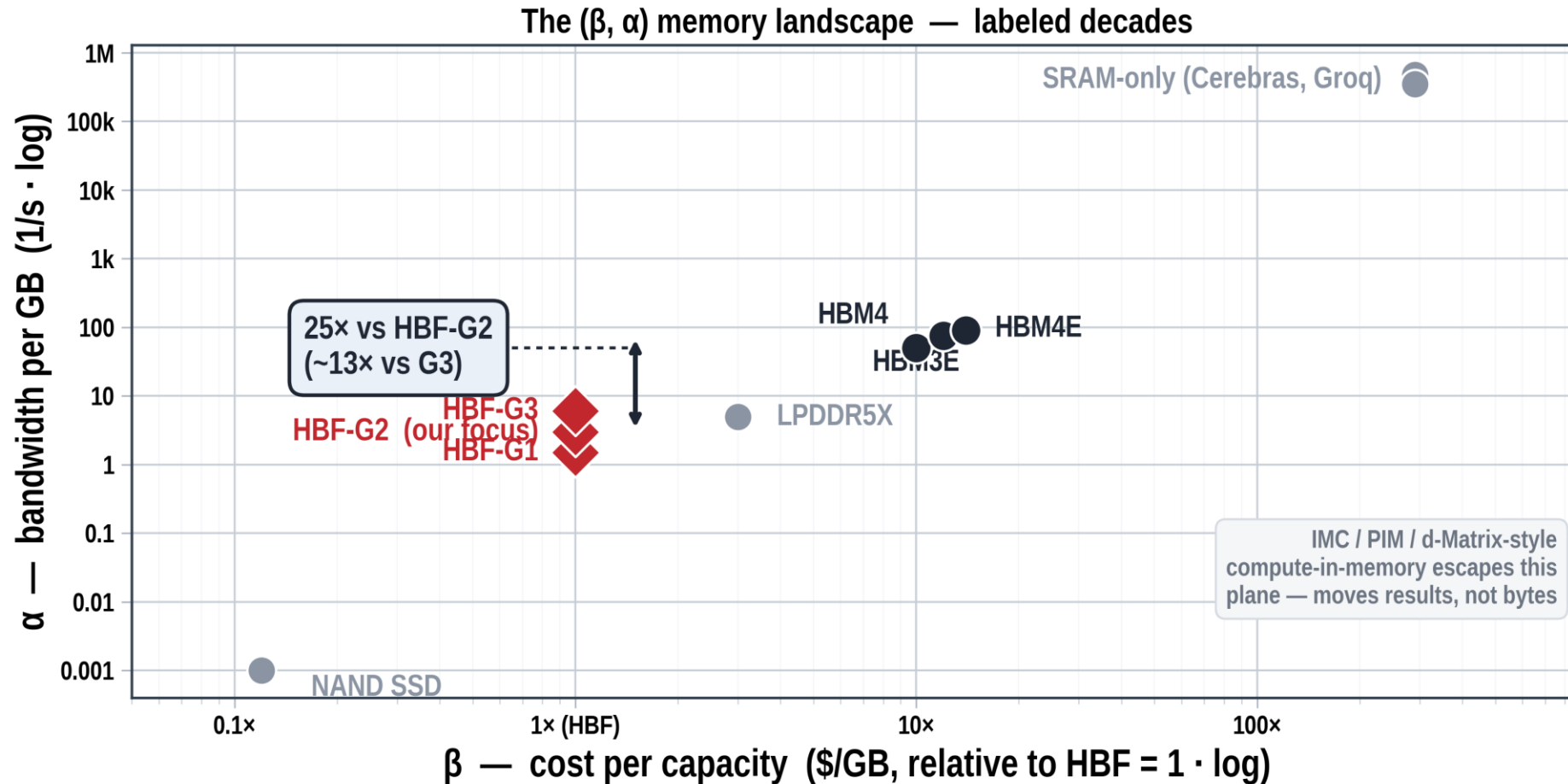
HBF cube	Grade 1	Grade 2	Grade 3
Max user BW	0.384 TB/s	1.536 TB/s	3.072 TB/s
UCle rate	8 GT/s	16 GT/s	32 GT/s
Capacity	8-high · 256 GiB	16-high · 512 GiB	16-high · 512 GiB
Access	64 B–4 KiB reads · 4 KiB writes · 4 KiB page		
Write endurance	Left open-ended		

8–16× the capacity of HBM — at the same cost

The cheapest \$/GB can be the most expensive \$/token.

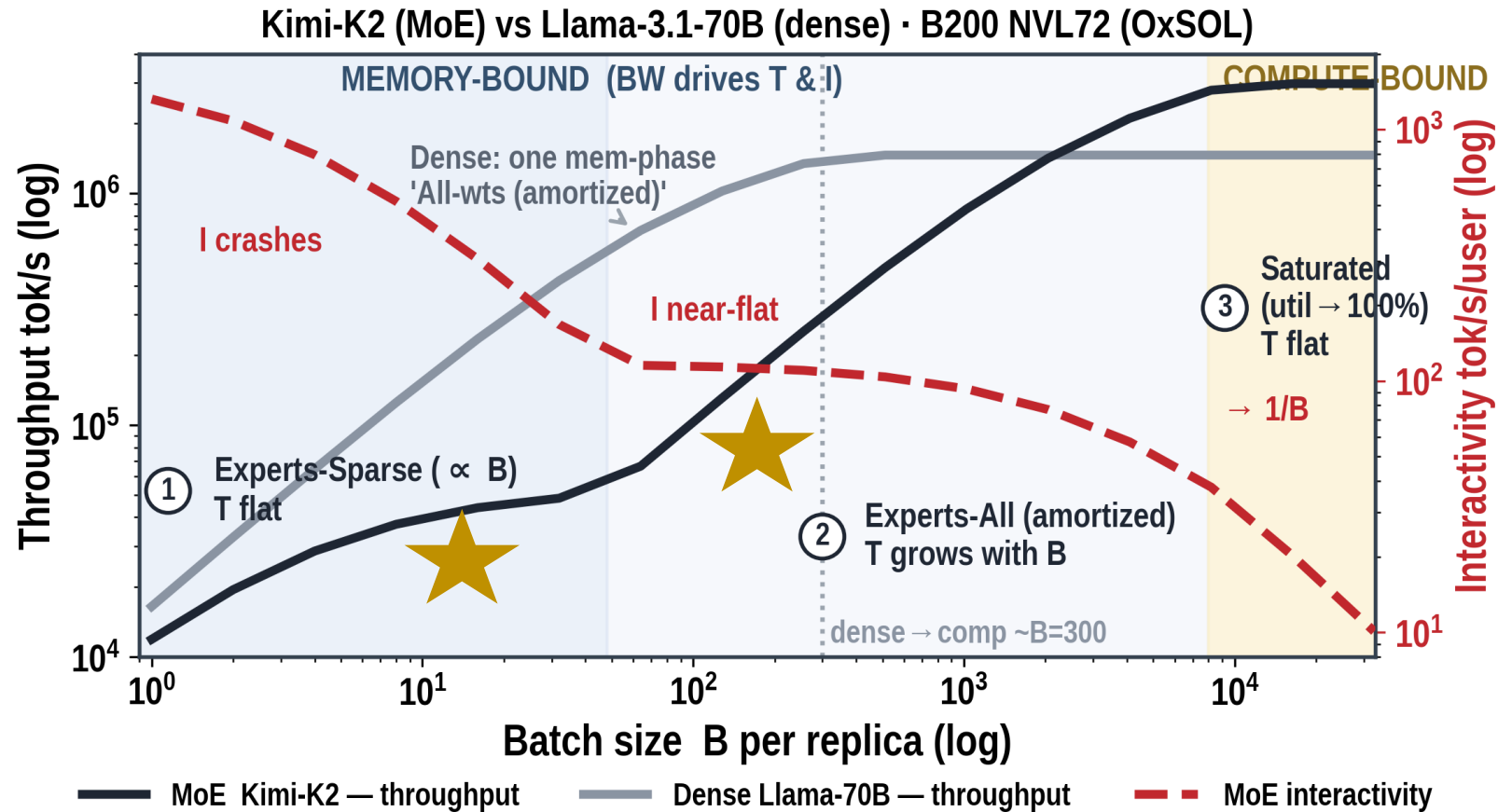
HBF — bargain or trap?

Memory technology — the (β, α) landscape



HBF is a capacity point (low α , low β) — not a cheaper HBM

Model personality — MoE vs Dense



in decode, per step:

$$b = \left[1 - \left(1 - \frac{k}{N} \right)^B \right] W + W_0 + B \cdot K$$

bytes read / step

$$I = BW/b$$

interactivity — tok/s/user

$$T = B \cdot I$$

throughput — tok/s

N = total experts · k = active / token
 B = batch (= users) · W = routed-expert wts
 W_0 = fixed wts (shared experts + dense)
 K = KV bytes / token

MoE opens a small-B capacity sub-phase — model held in full, low bandwidth is fine

The cost of serving — a token

$$\text{\$}_{\text{mem}} = \beta \cdot \max\left(C, \frac{\text{BW demand}}{\alpha}\right)$$

$$\text{BW demand} = l \cdot b$$

GB to HOLD (C)

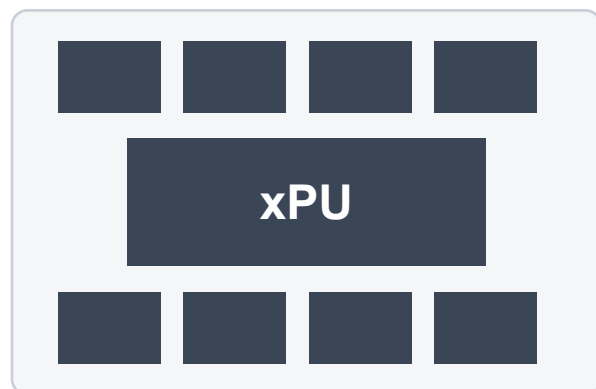
vs

GB to FEED (BW demand / α)

You aren't buying GB for dollars — you're buying the outcome of this equation

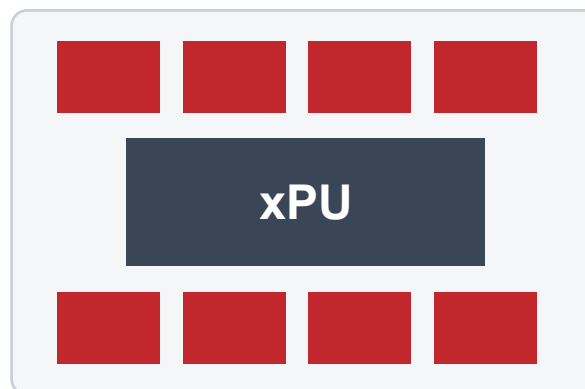
Few ways to deploy HBF

HBM-only (baseline)



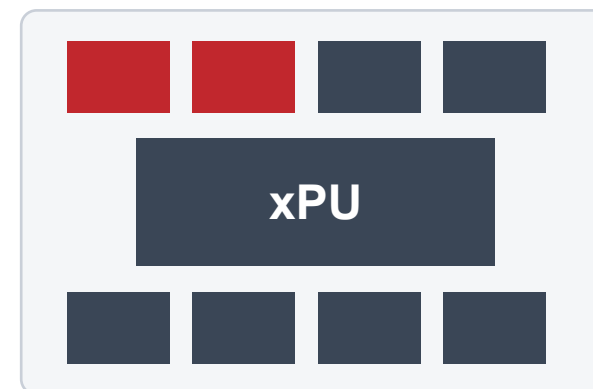
288 GB · 22.0 TB/s

(a) All-HBF



4,096 GB · 12.8 TB/s

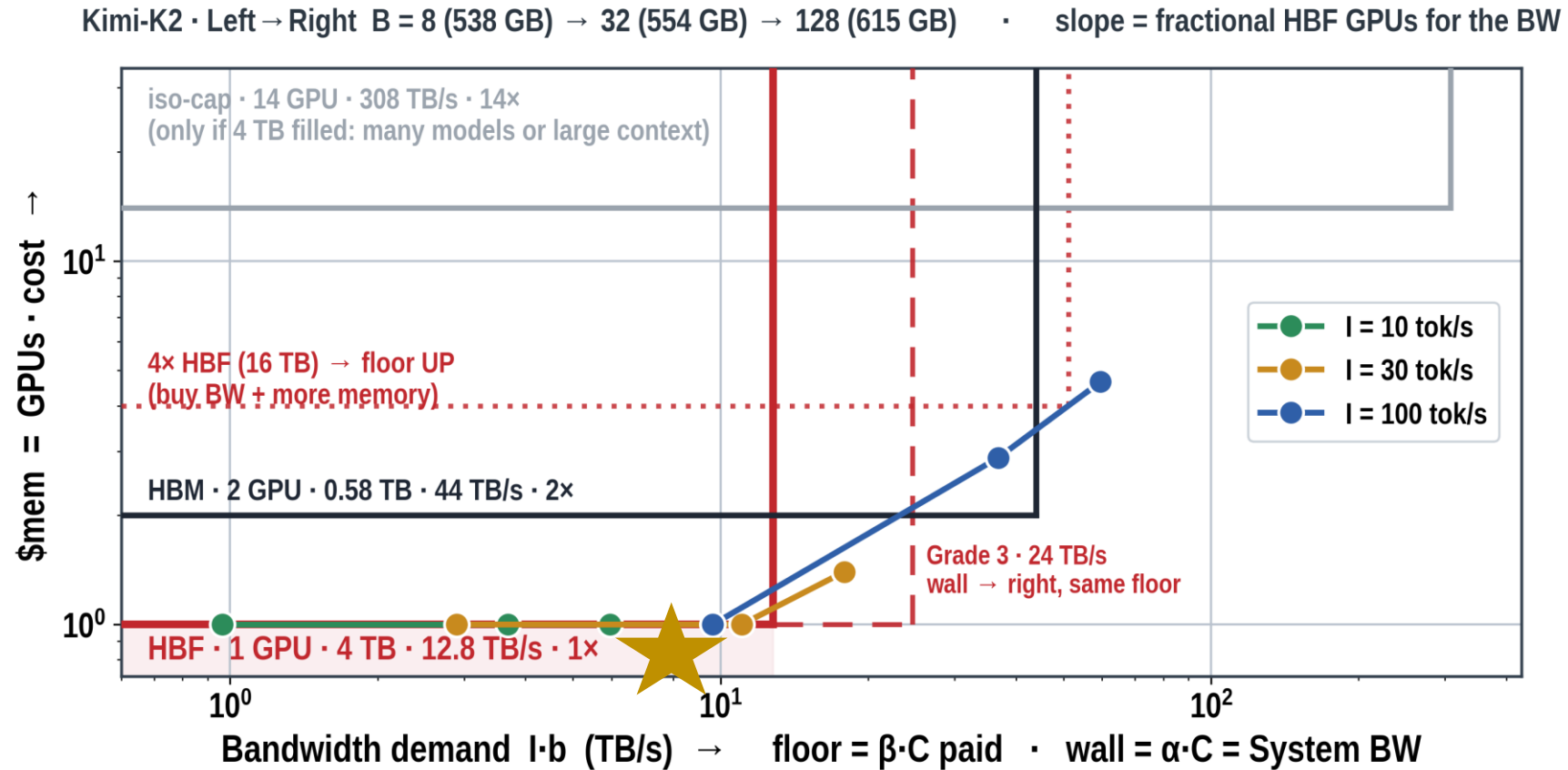
(b) 2×HBF + 6×HBM



1,240 GB · 19.7 TB/s peak
(eff. 19→4 TB/s across B)

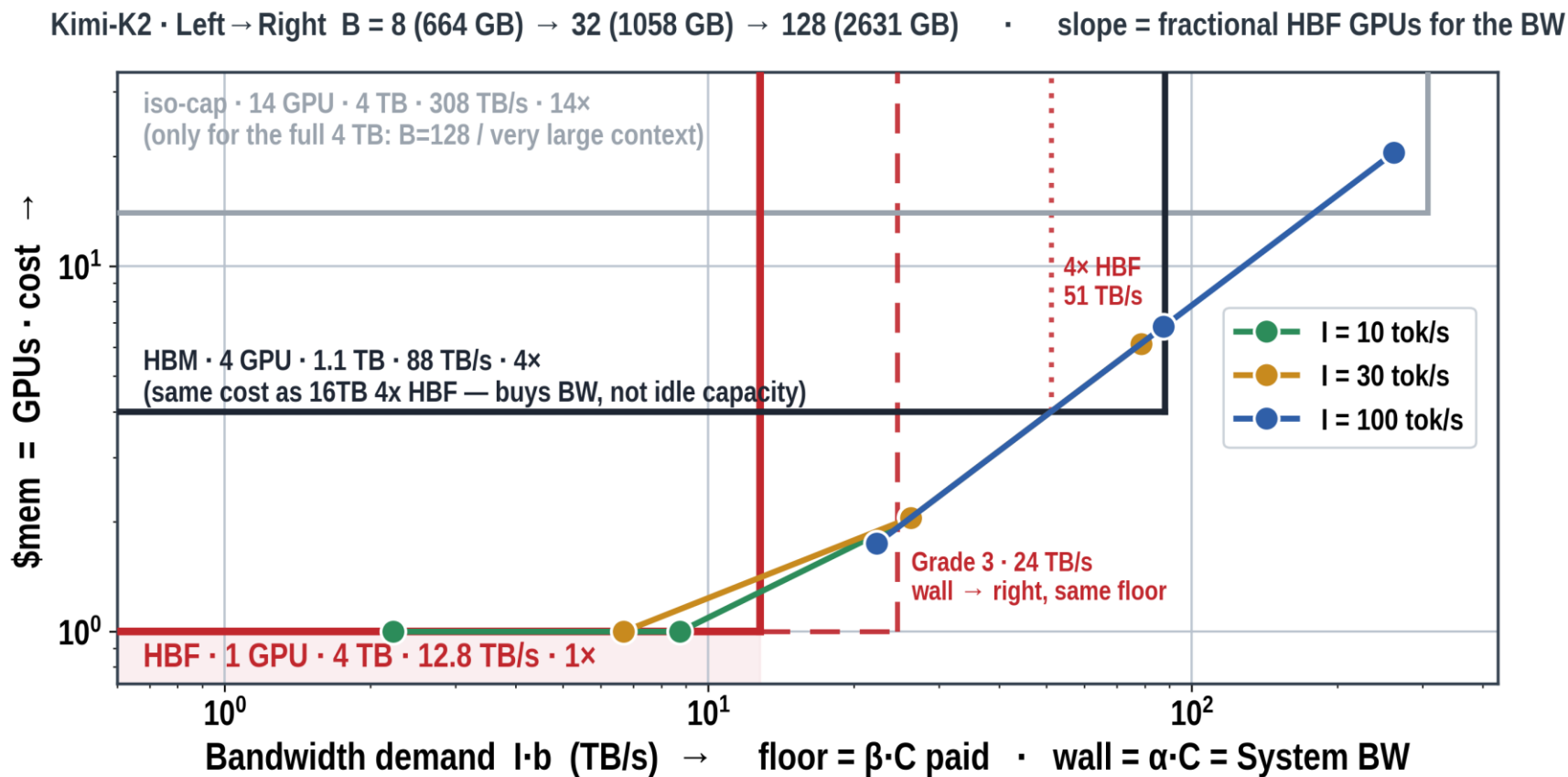
Same cost. More capacity. Less bandwidth.

All-HBF · short context (256/256)



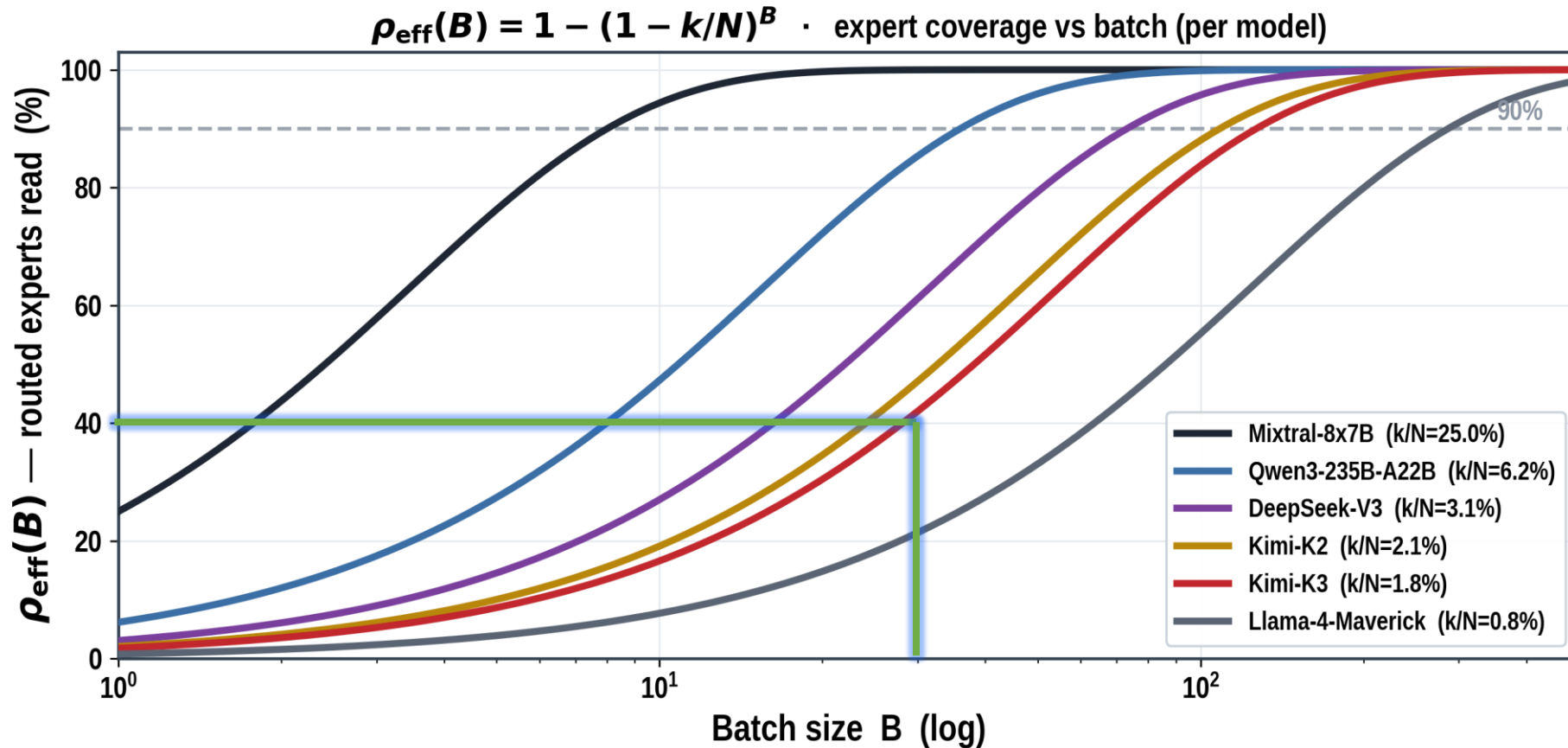
HBF wins on cost at low B but with 85% dead capacity — HBM pays well past the wall

All-HBF · long context (1M/1K)



Capacity wins only while $I \cdot b$ stays low — past that, HBM is the better buy

Mixed case · HBM as the hot-expert cache



Expert popularity flattens across mixed queries — caching pays only at low B , or when 'like' queries are batched together

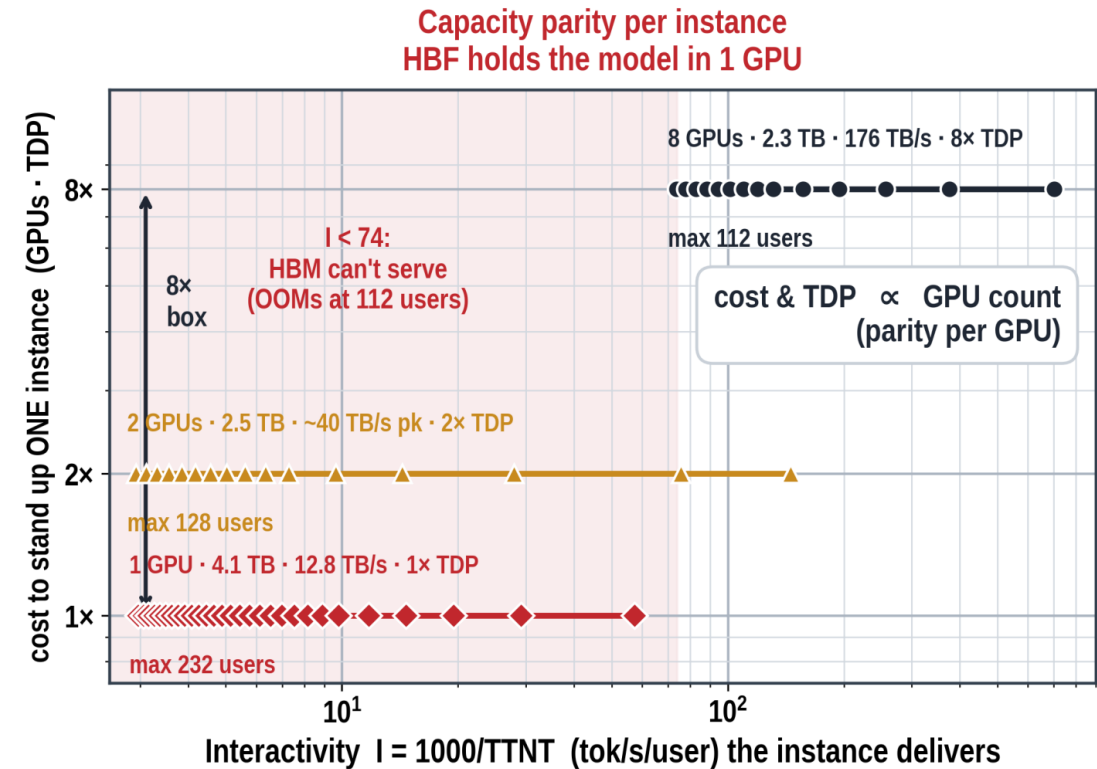
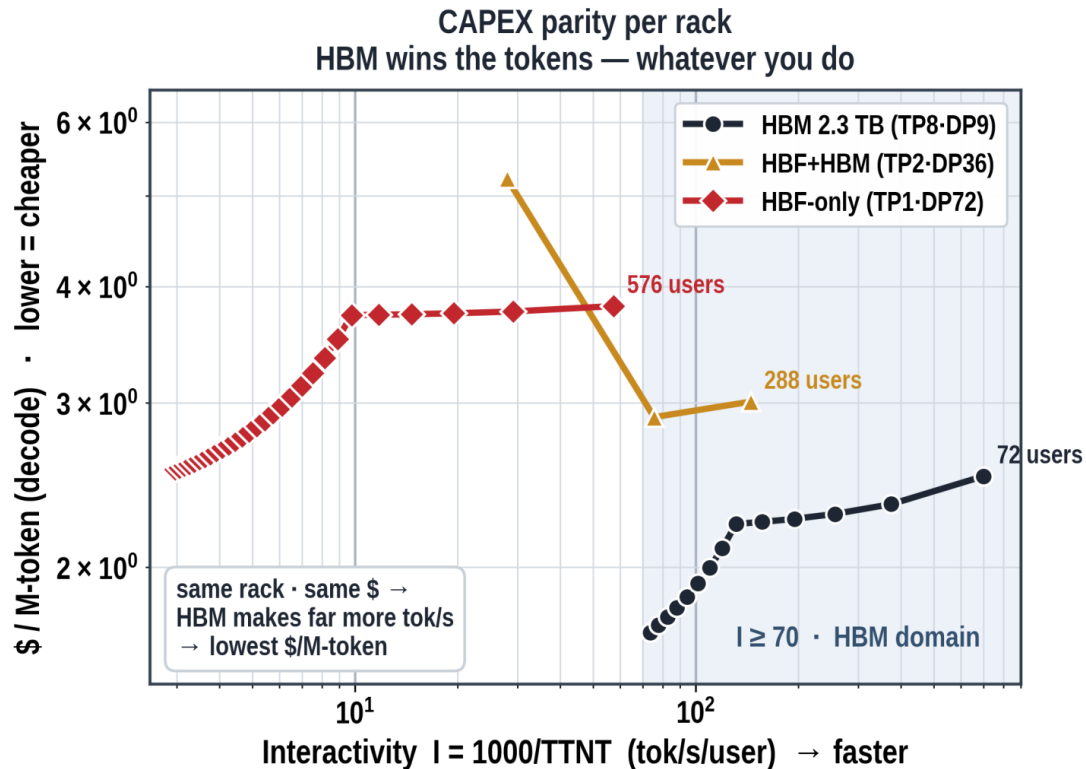
System & simulation setup — rack level

72-GPU rack	HBM-only TP8 · DP9	HBF-only TP1 · DP72	HBF + HBM TP2 · DP36
Mem capacity / DP	2.3 TB	4.1 TB	2.5 TB
Mem capacity · rack	20.7 TB (1×)	294.9 TB (14×)	89.3 TB (4.3×)
Aggregate BW	1,584 TB/s	922 TB/s (0.6×)	1,418 → 279 TB/s
Per-GPU BW	22.0 TB/s	12.8 TB/s	19.7 → 3.9 TB/s
Cost · power	1× · parity	1× · parity	1× · parity

Model: Kimi-K2 1T @ FP4 · Context 1M in / 1K out (long-context; + 32k/8k, 1k/8k) · Batch 1–512 per DP · TP fits capacity, DP fills the 72-GPU rack · Cost: 3-yr amortized (capex+opex), shown as ratios · decode-centric

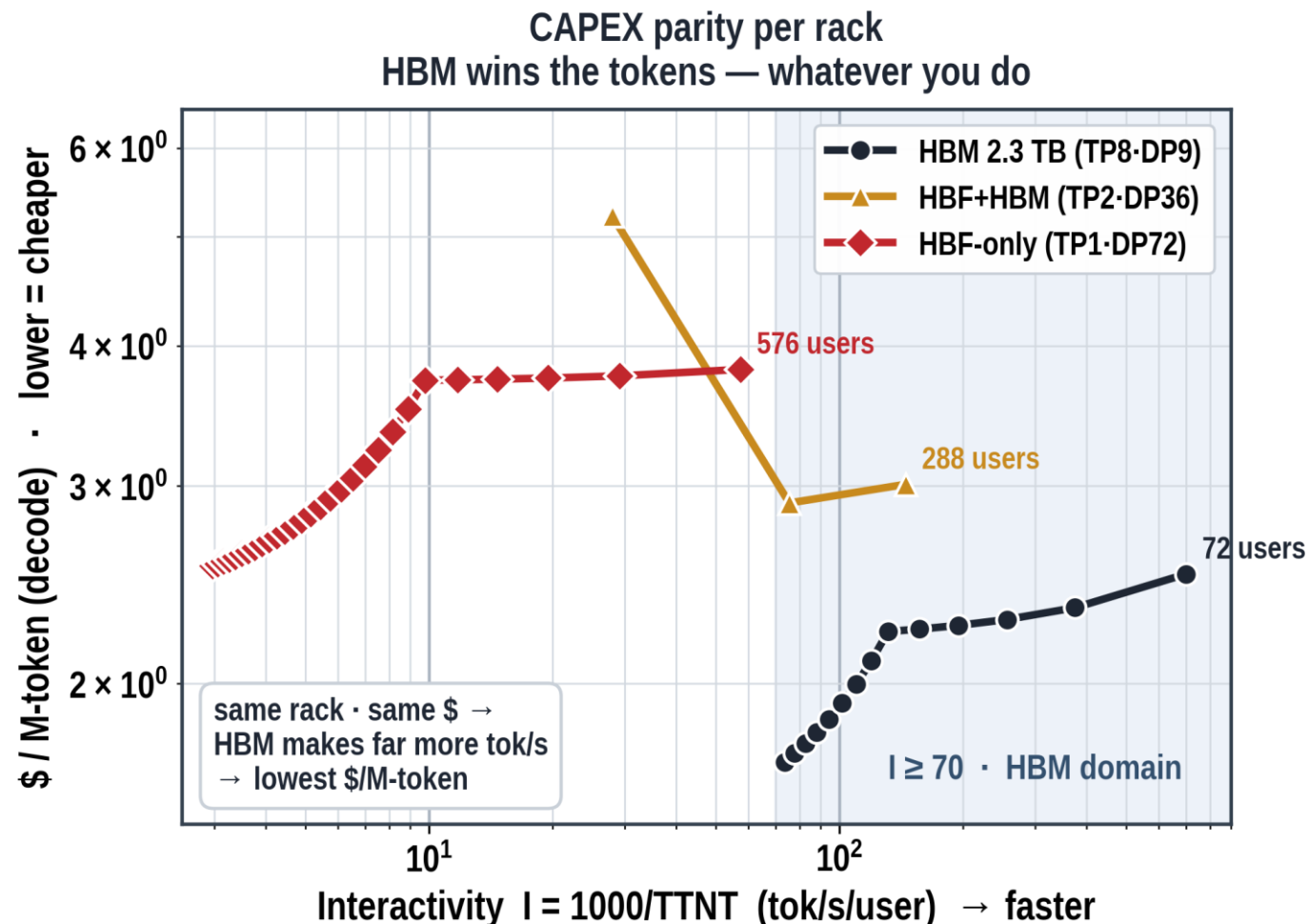
Same rack, same cost → HBF buys ~14× capacity for ~0.6× bandwidth

Rack \$/token vs single-server capacity



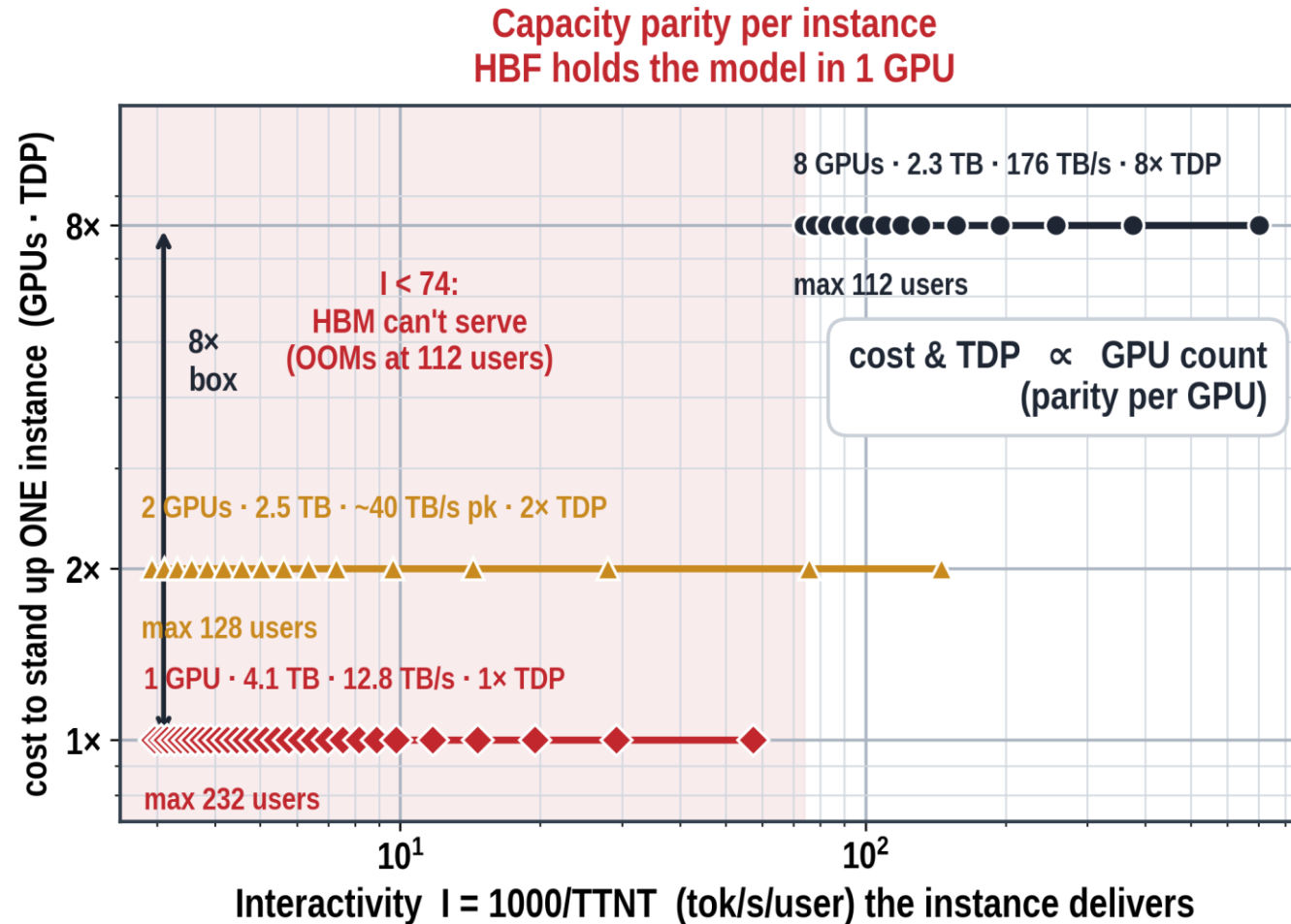
Cheaper \$/GB $\neq$ cheaper \$/token — HBM for the rack, HBF for the box: pick your scale

Rack \$/token vs single-server capacity



Cheaper \$/GB $\neq$ cheaper \$/token — HBM for the rack, HBF for the box: pick your scale

Rack \$/token vs single-server capacity



Cheaper \$/GB $\neq$ cheaper \$/token — HBM for the rack, HBF for the box: pick your scale

HBF spec — the software constraints

- Access chunks for max BW: 64 KB reads, 1 MB writes (64 KB aligned)
- HBF read/write through DMA — not designed for the GPU cache hierarchy
- Separate memory management when HBF and HBM are used together
- Power-on data retention ~24h at 85 °C — host-managed lifecycle required
- Careful host mgmt. to reach ~10-yr life or 100% endurance consumption
- Scratchpad SRAM cannot read/write the NAND blocks directly

Read-optimized, write-constrained, host-managed — placement is a software problem

Software ecosystem — inference engines

Production Serving Inference Engines

- High-throughput cloud deployments
- Continuous batching
- Paged KV memory and prefix caching
- Broad model support and portable across accelerators

Vendor-Optimized Inference Engines

- Peak-throughput on vendor's own silicon
- Silicon-specific optimizations owned by chip vendor

Production Serving

vLLM

SGLang

LMDeploy

Modular MAX

Vendor-Optimized Stacks

TensorRT-LLM

OpenVINO

AWS Neuron

Google
JetStream

vLLM is the production default — our focus

Where the memory goes — Kimi-K3

93% bytes are MoE expert weights (1.45 TB)

- **HBF** is better for MoE expert pool
- Write-once, read-cold

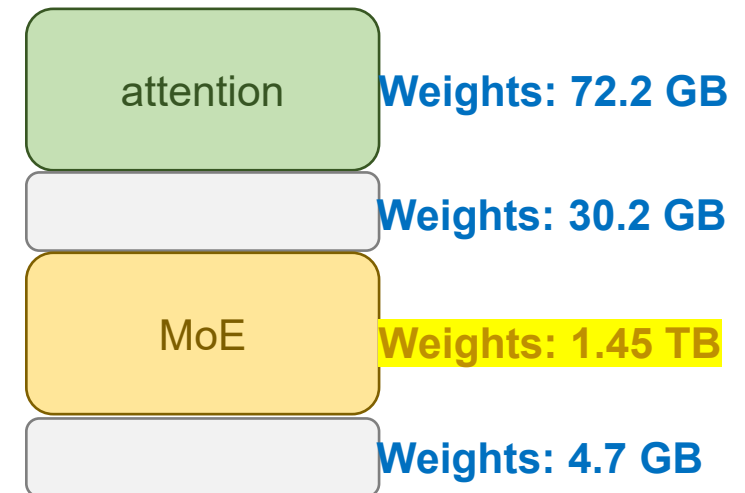
Remaining 7% weights storage (110 GB)

- **HBM** is better

KV-Cache size per 1M token sequence: 30GB

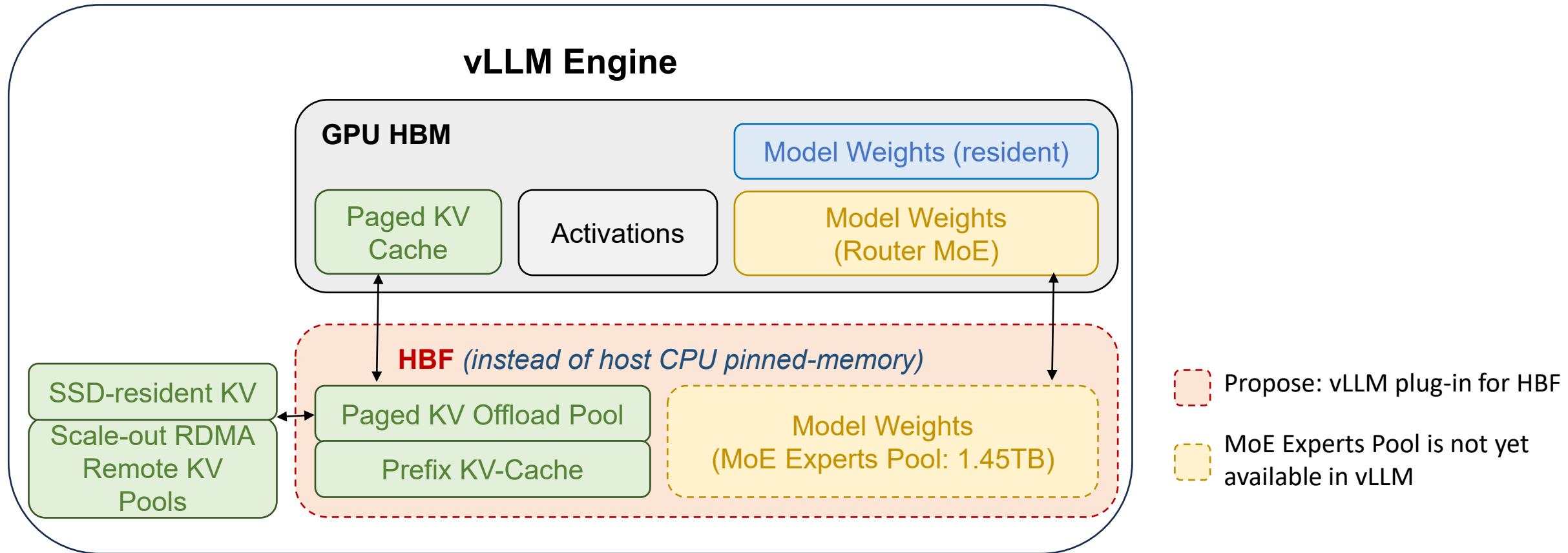
- **HBF** is better for KV-Cache offload pool + prefix-cache

Kimi K3 2.8T: Total Weights 1.56 TB



Weights & offloaded KV: huge, write-once, read-cold — low I-b, exactly HBF's capacity zone

Inside vLLM – The Memory Subsystem



HBF replaces host-DRAM pinned memory for the KV & MoE pools— with far more capacity

Attention sparsity — plays in HBF's zone

Long-context / multi-turn KV: store it all cheap, read only the top-k

Full attention



reads 100% / step
→ high BW → HBM

Sparse attention



top-k $\approx$ 1-2k rows
read this step

full KV — long context /
multi-turn → many GB, cold
→ held cheaply in HBF

stores 100% (cold)
reads only top-k / step

$\phi = \text{top-k/context} \approx 1-2\% \ll \phi^* = \alpha/l \rightarrow$ hold the whole KV cheap in HBF, read only the top-k

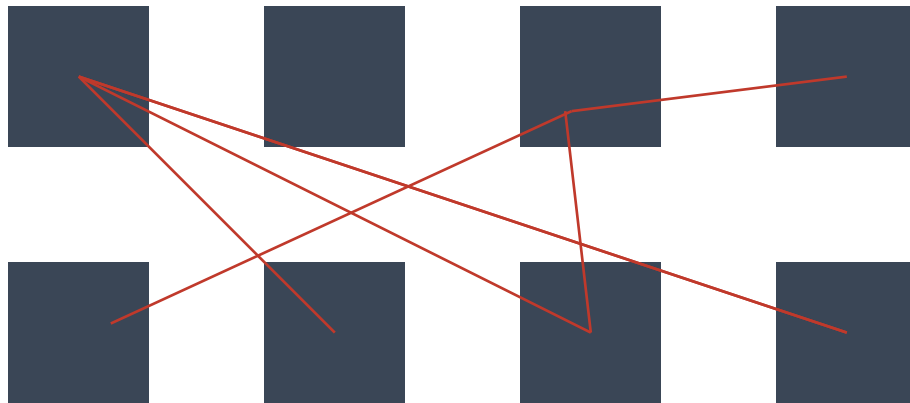
A genuine HBF fit — conditional on sparse-attention models (DSA · CSA · Kimi-Linear)

Ref: DeepSeek-V3.2 · DeepSeek-V4 · Tutti (2605.03375) · HiFC (endurance)

DSA = DeepSeek Sparse Attention · CSA = Compressed Sparse Attention

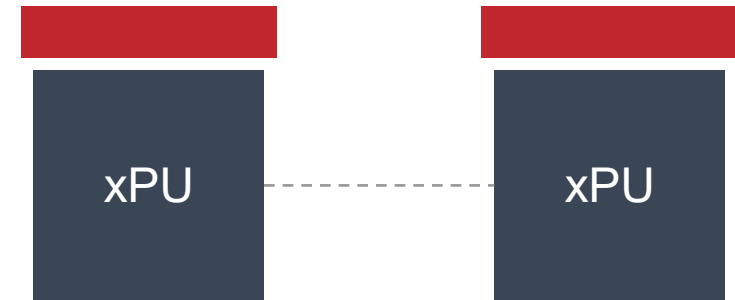
EP × HBF — capacity buys back the comm

Expert Parallelism



8 GPUs · experts sharded · all-to-all every layer

HBF capacity



2 nodes · experts local in HBF · little/no all-to-all

Cheap HBF capacity → fewer EP shards → less all-to-all comm

Takeaways — bargain or trap?

- 1 Judge any memory by one number: $\$mem = \beta \cdot \max(C, I \cdot b / \alpha)$; the figure of merit is $(\beta/\alpha) \cdot b$ —OR technology times the workload.
- 2 HBF wins in ONE zone — low bandwidth demand ($I \cdot b$): MoE at small B / low I , and long-context sparse KV. Dense and high- B stay on HBM.
- 3 For HBF to matter more: read BW must rise ($\alpha \uparrow$), the $\$/GB$ gap must stay wide ($\beta \downarrow$ vs HBM), and write endurance + write BW/latency must be handled.
- 4 Software prerequisite: an HBF allocator/backend, placement policy, async prefetch, and endurance telemetry — today's vLLM paths target CPU/LMCache, not HBF.

HBF: A precision instrument, not a hammer — bargain in its zone, trap outside it

What's next at OXMIQ

Multi-agent locality · prefetch-BW limits · memory config to workload

Agents to Atoms

We're hiring